Banka se třemi pracovníky

Zadání

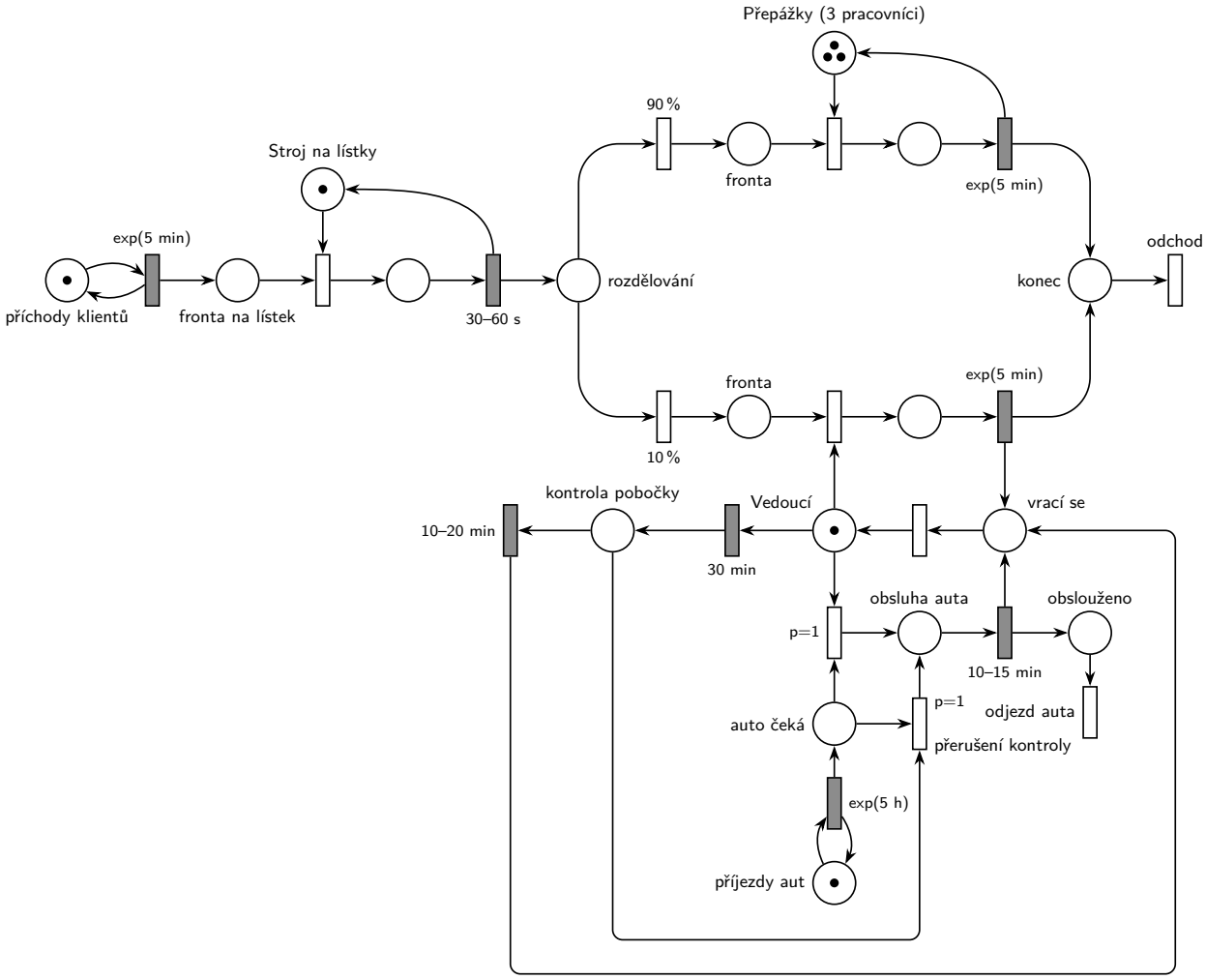
> Do bankovní pobočky přicházejí klienti v intervalech exponenciálně se středem 5 minut. V pobočce pracují tři pracovníci na přepážce a jeden vedoucí pobočky. Klient si po příchodu do pobočky vezme lístek ze stroje pro evidenci pořadí. Použití stroje zabere rovnoměrně 30–60 sekund. Deset procent klientů požaduje speciální poradenství, které vykonává vedoucí pobočky. Ostatní klienti jsou obsluhováni na přepážkách. Obsluha na přepážce i speciální poradenství trvají exponenciálně se středem 5 minut.
>
> Vedoucí pobočky je v počátečním stavu na své přepážce. Pokud je k jeho přepážce fronta klientů prázdná již přesně 30 minut, vydává se na kontrolu pobočky, která mu zabere rovnoměrně 10–20 minut. V intervalech exponenciálně se středem 5 hodin přijíždí do pobočky vozidlo s převozem hotovosti, které má pro vedoucího vysokou prioritu. Pokud momentálně neobsluhuje klienta, musí svou činnost přerušit a obsloužit auto. Pokud momentálně obsluhuje klienta, dokončí nejprve obsluhu klienta a pak započne obsluhu auta. Obsluha auta zabere rovnoměrně 10–15 minut. Po obsluze auta se vrací k přepážce.

Podobné úlohy: Banka a lupič, Spánek po nečinnosti, Automat na kávu – údržba po nečinnosti, Příklad s prioritou procesu, Kopírovací centrum.

Nápověda

- **Příchody** modeluje místo *příchody klientů* ve smyčce s `exp(5 min)`.
- **Stroj na lístky** je jedno zařízení (jeden stroj), před ním se čeká ve *frontě na lístek*; `30–60 s` vrací značku stroje. **Tři pracovníci** jsou sklad se 3 značkami, obsluha `exp(5 min)` vrací pracovníka.
- **Rozdělení** z místa *rozdělování*: okamžité přechody `90 %` nahoru na přepážky a `10 %` dolů k vedoucímu. Obě větve se slévají v místě *konec* a klient odchází přechodem *odchod*.
- **Vedoucí jako proces:** značka *Vedoucí* znamená „stojí u přepážky a nemá klienta“. Kromě obsazení klientem ji může vzít kontrola pobočky nebo auto, takže je to zároveň stav vedoucího. Ze všech tří činností (poradenství, kontrola, obsluha auta) se vedoucí vrací přes místo *vrací se* a okamžitý přechod do *Vedoucí*. Je to jen kvůli kresbě (místo *Vedoucí* by jinak mělo šest hran); v nulovém čase se tím nic nemění.
- **„Fronta prázdná již 30 minut“:** časovaný přechod `30 min` jen se vstupem z *Vedoucí*. Odpočet běží, dokud vedoucí stojí u přepážky bez klienta. Přijde-li klient, obsazení (okamžitý přechod) značku vezme a odpočet zanikne (slajd 139). Doba, kdy vedoucí obsluhuje, se do 30 minut nepočítá; doslovné „fronta je prázdná 30 minut i během obsluhy“ by bez inhibiční hrany nešlo nakreslit a smysl zadání je „vedoucí nemá co dělat“. Kontrola pobočky trvá `10–20 min`.
- **Auto je samostatná transakce** s vlastním generátorem: *příjezdy aut* ve smyčce s `exp(5 h)`, čeká v místě *auto čeká*.
- **Auto s vysokou prioritou:** dva okamžité přechody `p=1`. Jeden vezme volného vedoucího u přepážky, druhý *přeruší kontrolu pobočky* (vezme značku z *kontrola pobočky*, takže odpočet `10–20 min` zanikne). Když vedoucí obsluhuje klienta, auto čeká v místě *auto čeká*; po skončení obsluhy vyhraje `p=1` nad obsazením dalším klientem (`p=0`). Probíhající obsluhu klienta auto nepřerušuje.
- Obsluha auta trvá `10–15 min`, pak auto odjíždí přes *obslouženo* přechodem *odjezd auta*. Přerušená kontrola se už nedokončuje: po obsluze auta se vedoucí vrací k přepážce.

Petriho síť



Časté chyby

- Časovač `30 min` nezávislý na vedoucím (vlastní generátor). Pak vedoucí odejde na kontrolu i od klienta nebo se odpočet nezruší, když klient přijde.
- Auto bez p=1: po skončení obsluhy klienta by vedoucího mohl obsadit další klient.
- Auto, které přeruší i probíhající poradenství; zadání chce obsluhu klienta nejprve dokončit.
- Chybějící přerušení kontroly: auto by čekalo až 20 minut, než se vedoucí vrátí.
- Přepážky jako tři samostatná zařízení s náhodným výběrem; stačí jeden sklad se 3 značkami.

SIMLIB

Model je izomorfní k Petriho síti výše: každé místo, přechod a značka mají svůj protějšek v kódu.

```
// Banka se třemi pracovníky – model v SIMLIB/C++
#include "simlib.h"

const double HOD = 60;                  // časová jednotka: minuta

Facility Stroj("Stroj na lístky");      // místo Stroj na lístky (1 značka)
Store Prepazky("Přepážky", 3);          // místo Přepážky (3 značky)
Facility Vedouci("Vedoucí");            // značka vedoucího; kdo ji drží, tam
 ↪  vedoucí je
Histogram Doba("Doba klienta v bance [min]", 0, 2, 20);
int kontroly = 0, prerusene = 0, auta = 0;
```

```
class Necinnost : public Event {        // přechod 30 min na místě Vedoucí
  void Behavior() override;
};
Necinnost *necinnost;

void Uvolneni() {                      // vedoucí se vrací k přepážce (místo vrací
 ↪  se)
  if (!Vedouci.Busy())                 // nikdo nečeká: místo Vedoucí, začíná
   ↪  odpočet 30 min
    necinnost->Activate(Time + 30);
}

class Kontrola : public Process {       // místo kontrola pobočky
 public:
  bool prerusena = false;
  void Behavior() override;
};
Kontrola *kontrola = nullptr;

void Kontrola::Behavior() {
  Seize(Vedouci);                      // vezme značku vedoucího (je volný)
  kontroly++;
  Wait(Uniform(10, 20));               // přechod 10–20 min (přerušení ho ukončí
 ↪  dřív)
  kontrola = nullptr;
  Release(Vedouci);                    // čekající auto (priorita 1) dostane
 ↪  vedoucího
  if (!prerusena)
    Uvolneni();
}

void Necinnost::Behavior() {
  kontrola = new Kontrola;
  kontrola->Activate();
}

class Klient : public Process {         // transakce = značka klienta
  void Behavior() override {
    double prichod = Time;
    Seize(Stroj);                      // fronta na lístek + obsazení stroje
    Wait(Uniform(0.5, 1));             // 30–60 s, vrací stroj
    Release(Stroj);
    if (Random() < 0.9) {              // 90 % na přepážky
      Enter(Prepazky, 1);
      Wait(Exponential(5));
      Leave(Prepazky, 1);
    } else {                           // 10 % k vedoucímu
      necinnost->Passivate();          // obsazení vezme značku, odpočet 30 min
 ↪  zanikne
      Seize(Vedouci);
      Wait(Exponential(5));
      Release(Vedouci);
      Uvolneni();
    }
    Doba(Time - prichod);
  }                                    // místo konec + přechod odchod
};

class Auto : public Process {           // transakce = auto s hotovostí
 public:
  Auto() : Process(1) {}               // priorita procesu 1: obsazení p=1 před
 ↪  klienty
```

```
  void Behavior() override {
    auta++;
    if (kontrola != nullptr) {          // přerušení kontroly (p=1)
      kontrola->prerusena = true;
      kontrola->Activate();
      prerusene++;
    }
    necinnost->Passivate();
    Seize(Vedouci);                    // místo auto čeká; klienta vedoucí nejdřív
 ↪  dokončí
    Wait(Uniform(10, 15));              // místo obsluha auta, přechod 10–15 min
    Release(Vedouci);
    Uvolneni();
  }                                     // místo obslouženo + přechod odjezd auta
};

class Prichody : public Event {         // místo příchody klientů ve smyčce s
 ↪  exp(5 min)
  void Behavior() override {
    (new Klient)->Activate();
    Activate(Time + Exponential(5));
  }
};

class PrijezdyAut : public Event {      // místo příjezdy aut ve smyčce s exp(5 h)
  void Behavior() override {
    (new Auto)->Activate();
    Activate(Time + Exponential(5 * HOD));
  }
};

int main() {
  Init(0, 1000 * HOD);                  // modelový čas v minutách: 1000 hodin
 ↪  provozu
  necinnost = new Necinnost;
  Uvolneni();                           // vedoucí je na začátku u přepážky
  (new Prichody)->Activate();
  (new PrijezdyAut)->Activate(Exponential(5 * HOD));
  Run();
  Print("Kontroly pobočky: %d, z toho přerušené autem: %d, auta: %d\n",
        kontroly, prerusene, auta);
  Stroj.Output();
  Prepazky.Output();
  Vedouci.Output();
  Doba.Output();
}
```

Petriho síť	SIMLIB
příchody klientů ⇄ exp(5 min)	událost Prichody, znovu za Exponential(5)
značka klienta	proces Klient
Stroj na lístky (1 značka), obsazení, 30–60 s	Seize(Stroj); Wait(Uniform(0.5, 1));
vrací značku	Release(Stroj);
90 % / 10 %	if (Random() < 0.9)
Přepážky (3 značky), exp(5 min)	Store Prepazky("Přepážky", 3), Enter / Wait / Leave
Vedoucí + poradenství exp(5 min)	Facility Vedouci, Seize / Wait(Exponential(5)) / Release
vrací se → *Vedoucí*	funkce Uvolneni() po každém Release(Vedouci)
30 min na místě *Vedoucí*	událost Necinnost: Activate(Time + 30) při uvolnění, Passivate() při obsazení

Petriho síť	SIMLIB
kontrola pobočky → `10–20 min`	proces `Kontrola`: `Seize(Vedouci);` `Wait(Uniform(10, 20));` `Release(Vedouci);`
příjezdy aut ⇄ `exp(5 h)`, *auto čeká*	událost `PrijezdyAut`, proces `Auto`
obsazení vedoucího autem (`p=1`)	`Auto() : Process(1)` (priorita procesu řadí frontu, obsluhu klienta nepřeruší)
přerušení kontroly (`p=1`)	`kontrola->Activate()` ukončí její `Wait`, `Release` předá vedoucího autu
obsluha auta → `10–15 min`	`Wait(Uniform(10, 15));` `Release(Vedouci);`
konec + odchod, *obslouženo + odjezd auta*	konec metody `Behavior()`

Kontrola simulací. Petriho síť: stroj, přepážky (= 3) i vedoucí (*Vedoucí* + poradenství + kontrola + obsluha auta + *vrací se* = 1) se vždy vrátí, fronty zůstávají krátké (`make check`). SIMLIB model: za 1000 hodin byl stroj vytížen na 15 % (teoreticky 1/5 klienta za minutu × 0,75 min), přepážky v průměru 0,90 ze 3 (0,9 · 1/5 × 5 min = 0,9). Vedoucí šel 887× na kontrolu (má jen 10 % klientů, takže je často 30 minut bez práce), přijelo 185 aut (teoreticky 200) a 33 z nich přerušilo kontrolu. Vedoucí byl obsazen 35 % času: kontroly asi 22 %, klienti 10 %, auta 4 %.

Původ a zdroje

Původ: půlsemestrální písemka 2013/14, varianta A, 6 bodů (k příkladu patřila ještě otázka z teorie za 4 body).

Původní znění:

> Do bankovní pobočky přicházejí klienti v intervalech daných exponenciálním rozložením se středem 5 minut. V pobočce pracují tři pracovníci na přepážce a jeden vedoucí pobočky. Klient si po příchodu do pobočky vezme lístek ze stroje pro evidenci pořadí. Použití stroje zabere 30-60 sekund rovnoměrně. Deset procent klientů požaduje speciální poradenství, které vykonává vedoucí pobočky. Ostatní klienti jsou obsluhováni na přepážkách. Obsluha na přepážce i speciální poradenství trvají dobu danou exponenciálním rozložením se středem 5 minut.
>
> Vedoucí pobočky je v počátečním stavu na své přepážce. Pokud je k jeho přepážce fronta klientů prázdná již 30 minut, vydává se na kontrolu pobočky, která mu zabere 10-20 minut. V intervalech daných exponenciálním rozložením se středem 5 hodin přijíždí do pobočky vozidlo s převozem hotovosti, které má pro vedoucího vysokou prioritu. Pokud momentálně neobsluhuje klienta, musí svou činnost přerušit a obsloužit auto. Pokud momentálně obsluhuje klienta, dokončí nejprve obsluhu klienta a pak započne obsluhu auta. Obsluha auta zabere 10-15 minut. Po obsluze auta se vrací k přepážce.
>
> *Zadaný systém modelujte formou Petriho sítě s vyznačením všech potřebných parametrů.*

Zdroje:

- `Fitcheats/Půlsemestrálky/Zadání z minulých let/2013-14.pdf`, str. 1 (zadání) a str. 3 (hlavička písemky s bodováním)
- `Fitcheats/Skripta/FIT - Studentská skripta 2015-16.pdf`, str. 48–49 (zadání zapsané po paměti a studentské řešení s komentáři)

Existující řešení:

- **Studentská skripta (str. 49), fotka ručního řešení:** stejná struktura: 90 %/10 %, přepážky jako místo se 3 značkami, vedoucí jako místo s 1 značkou, cyklus vedoucí → `30min` → kontrola → `10-20min` → vedoucí, auto `EXP(5h)` a dva přechody P=1 (auto + vedoucí, auto + místo kontroly) do obsluhy auta `10-15min`, výstupy „KLIENT ODCHÁZÍ“ a „AUTO ODJÍŽDÍ“. Výhrady:
  - Časované přechody nejsou graficky odlišené od okamžitých, poznají se jen podle připsaného času.

  - Za každou obsluhou je navíc místo a samostatný přechod uvolnění. Je to správně (plný tvar ze slajdu 154), jen delší; komentář pod řešením: „Souhlasím že ty místa za přechody s exp(5min) jsou navíc.“
  - Část kresby u stroje na lístky je přeškrtaná a špatně čitelná.
- Zadání ve skriptech (str. 48) je zapsané po paměti a zkrácené (chybí např. „dokončí nejprve obsluhu klienta“); platí text z `2013-14.pdf`.

Reference

- Jak síť funguje
- Notace a vzory
- Seznam úloh